

DOCUMENTO DE DISEÑO Y ARQUITECTURA TÉCNICA
CIVICVIEW: PLATAFORMA WEB INTEGRADA DE INFORMACIÓN
CIUDADANA EN TIEMPO REAL

AUTOR: JEYSON ANDRÉS ORTIZ MENDOZA

PROGRAMA: TECNÓLOGO EN ANÁLISIS Y DESARROLLO DE SOFTWARE
INSTITUCIÓN: SENA - CENTRO AGROEMPRESARIAL Y DESARROLLO
PECUARIO DEL HUILA

FECHA: DICIEMBRE 2025

VERSIÓN: 1.0

FASE: PLANIFICACIÓN Y DISEÑO (FASE 1)

TABLA DE CONTENIDO

1. Resumen Ejecutivo.
2. Definición de Requisitos.
3. Arquitectura del Sistema.
4. Diseño de Base de Datos.
5. Investigación y Análisis de APIs.
6. Diseño de la Experiencia de Usuario (UX).
7. Diseño de la Interfaz de Usuario (UI).
8. Stack Tecnológico.
9. Plan de Implementación.
10. Anexos.

1. RESUMEN EJECUTIVO

CivicView es una aplicación web progresiva (PWA) diseñada para centralizar información ciudadana relevante de Bogotá en un único dashboard intuitivo. Este documento presenta la arquitectura técnica, el diseño del sistema y las especificaciones funcionales que guiarán el desarrollo del proyecto durante las siguientes fases.

1.1 Objetivo del Documento

Establecer las bases técnicas y de diseño para el desarrollo de CivicView, definiendo la arquitectura del sistema, las tecnologías a utilizar, las integraciones necesarias y la experiencia de usuario propuesta.

1.2 Alcance

Este documento cubre:

- Especificación detallada de requisitos funcionales y no funcionales
- Arquitectura técnica del sistema (frontend, backend, APIs)
- Diseño de base de datos y modelo de caché
- Análisis y documentación de APIs externas
- Propuesta de diseño UX/UI
- Stack tecnológico y justificación de selección

2. DEFINICIÓN DE REQUISITOS

2.1 Requisitos Funcionales

RF-001: Módulo de Pico y Placa

Descripción: El sistema debe calcular y mostrar las restricciones de Pico y Placa vigentes para el día actual y el siguiente.

Criterios de Aceptación:

- Precisión del 100% basada en la normativa de la Secretaría de Movilidad de Bogotá
- Actualización automática a las 00:00 horas
- Indicador visual claro (puede circular / no puede circular)
- Visualización de los dígitos restringidos

Prioridad: Alta

Complejidad: Media

RF-002: Integración de Datos Meteorológicos

Descripción: El sistema debe mostrar información del clima actual y pronóstico extendido para Bogotá.

Criterios de Aceptación:

- Temperatura actual con precisión de $\pm 2^{\circ}\text{C}$
- Condición climática (soleado, nublado, lluvia, etc.)
- Probabilidad de precipitación
- Pronóstico para los próximos 3-7 días
- Actualización cada hora

Prioridad: Alta

Complejidad: Baja (API externa)

RF-003: Información de Calidad del Aire

****Descripción:**** El sistema debe mostrar el Índice de Calidad del Aire (ICA/AQI) actual de Bogotá.

Criterios de Aceptación:

- Valor numérico del ICA
- Categorización (Bueno, Moderado, Dañino, etc.)
- Código de colores según estándar internacional
- Recomendaciones básicas según nivel
- Actualización cada hora

Prioridad: Alta

Complejidad: Baja (API externa)

RF-004: Mapa Interactivo con Puntos de Interés

Descripción: El sistema debe mostrar un mapa interactivo de Bogotá con puntos de interés cívico.

Criterios de Aceptación:

- Visualización de al menos 500 puntos de interés simultáneos
- Categorías: ciclovías, parques, bibliotecas, centros culturales
- Posibilidad de zoom y navegación
- Filtros por categoría
- Información básica al hacer clic en cada punto

Prioridad: Media

Complejidad: Alta

RF-005: Listado de Eventos Culturales

Descripción: El sistema debe mostrar eventos culturales y recreativos disponibles en Bogotá.

Criterios de Aceptación:

- Lista de eventos del día actual
- Información básica: nombre, ubicación, hora
- Filtro por fecha
- Actualización diaria

Prioridad: Media

Complejidad: Media

RF-006: Diseño Responsive

Descripción: La interfaz debe adaptarse a diferentes dispositivos y resoluciones.

Criterios de Aceptación:

- Visualización óptima en móvil (375px+)
- Visualización óptima en tablet (768px+)
- Visualización óptima en desktop (1024px+)
- Sin pérdida de funcionalidad en ningún dispositivo

Prioridad: Alta

Complejidad: Media

2.2 Requisitos No Funcionales

RNF-001: Performance

- Tiempo de carga inicial:< 3 segundos (objetivo: 2 segundos)
- Tiempo de respuesta de APIs:< 2 segundos por petición
- First Contentful Paint (FCP): < 1.5 segundos
- Time to Interactive (TTI): < 3 segundos

RNF-002: Disponibilidad

- Uptime mínimo: 99% mensual (objetivo: 99.5%)
- Tolerancia a fallos:La aplicación debe seguir funcionando parcialmente si una API externa falla

RNF-003: Compatibilidad

- Navegadores soportados: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
- Sistemas operativos móviles: iOS 13+, Android 8+
- Cobertura objetivo: 95% de dispositivos del mercado actual

RNF-004: Usabilidad

- Tiempo promedio de interacción:1-2 minutos por sesión
- Curva de aprendizaje: El usuario debe poder utilizar la aplicación sin tutorial
- Accesibilidad: Cumplimiento WCAG 2.1 nivel AA

RNF-005: Seguridad

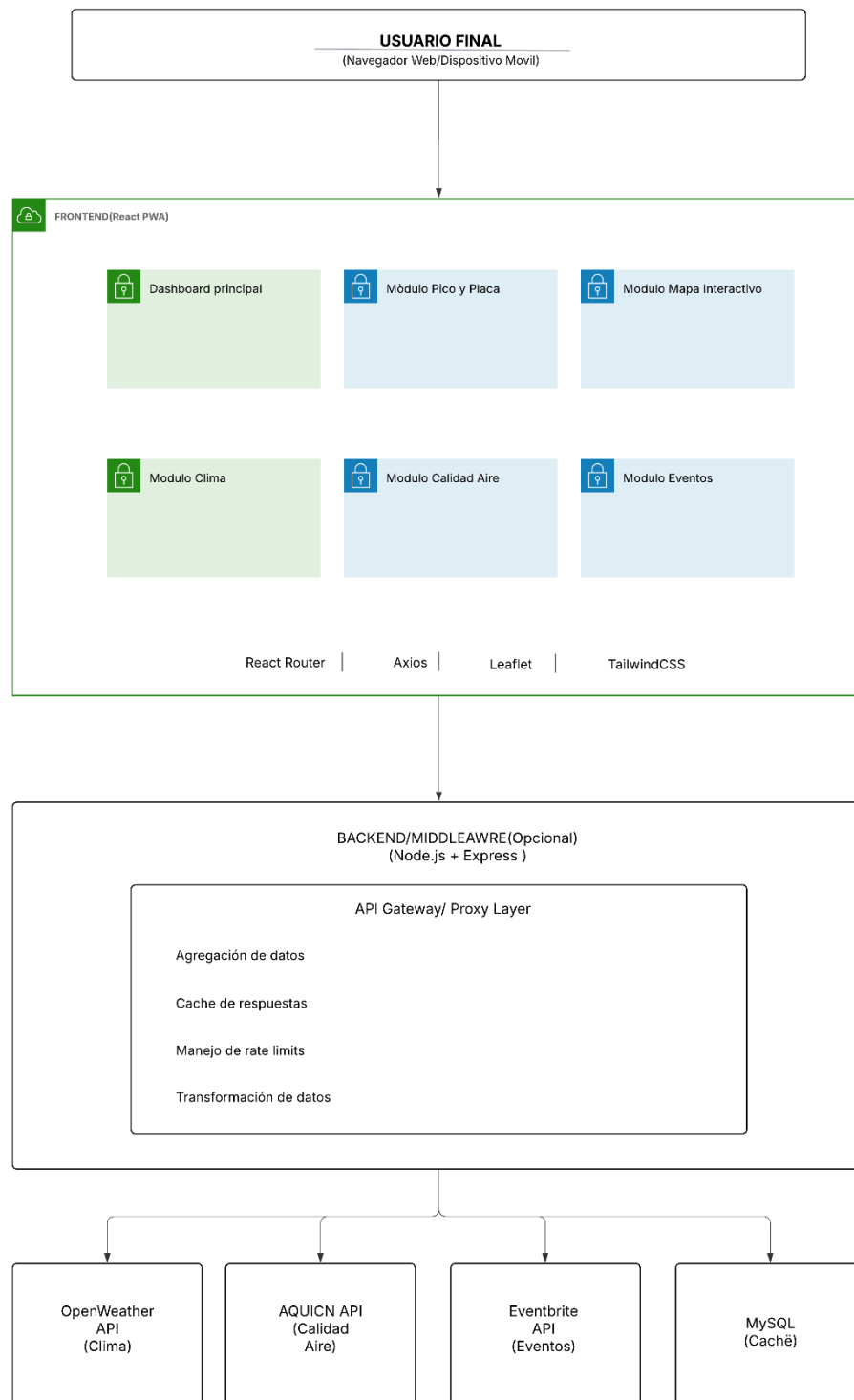
- Protocolo: HTTPS obligatorio
- Protección XSS: Sanitización de datos de entrada
- CORS: Configuración correcta de políticas de origen cruzado
- API Keys: Almacenamiento seguro en variables de entorno

RNF-006: Mantenibilidad

- Cobertura de código: Documentación inline mínima del 70%
- Modularidad: Componentes reutilizables y desacoplados
- Versionado: Control de versiones con Git (commits descriptivos)

3. ARQUITECTURA DEL SISTEMA

3.1 Diagrama de Arquitectura General



3.2 Arquitectura Frontend (React)

3.2.1 Estructura de Componentes

src/

```
|— components/
|   |— common/
|   |   |— Header.jsx
|   |   |— Footer.jsx
|   |   |— Card.jsx
|   |   |— Loading.jsx
|   |   └─ ErrorBoundary.jsx
|   |
|   |— dashboard/
|   |   |— Dashboard.jsx
|   |   |— PicoPlaca.jsx
|   |   |— Weather.jsx
|   |   |— AirQuality.jsx
|   |   |— Events.jsx
|   |   └─ MapWidget.jsx
|   |
|   └─ map/
|       |— InteractiveMap.jsx
|       |— MapMarker.jsx
|       └─ MapLegend.jsx
|
|— services/
|   |— weatherService.js
|   |— airQualityService.js
|   |— eventsService.js
|   └─ picoPlacaService.js
```



```
|
|
|— hooks/
|   |— useWeather.js
|   |— useAirQuality.js
|   └— useEvents.js
|
|
|— utils/
|   |— dateHelpers.js
|   |— apiHelpers.js
|   └— constants.js
|
|
|— context/
|   └— AppContext.jsx
|
|
|— styles/
|   └— tailwind.css
|
|
|— App.jsx
└— index.jsx
```

3.2.2 Patrón de Diseño

Patrón Principal: Composición de Componentes

Manejo de Estado: React Hooks (useState, useEffect, useContext)

Fetching de Datos: Custom Hooks + Axios

Routing: React Router (si se requieren múltiples vistas)

3.3 Arquitectura Backend (Opcional - Node.js)

El backend actuará como middleware/proxy entre el frontend y las APIs externas.

3.3.1 Responsabilidades del Backend

1. **API Gateway:** Centralizar llamadas a APIs externas
2. **Caché:** Reducir llamadas redundantes (ahorro de cuota)

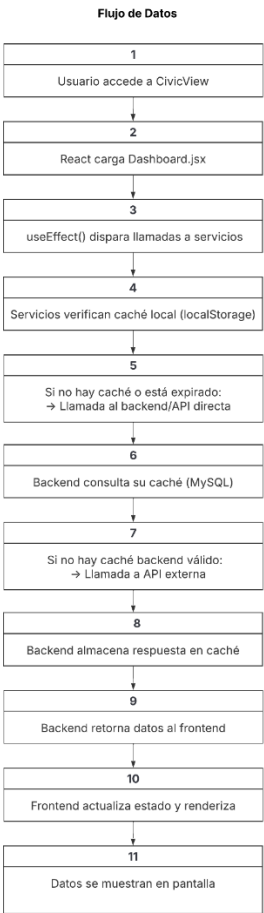
3. **Rate Limiting:** Controlar número de peticiones
4. **Transformación de Datos:** Normalizar respuestas de diferentes APIs
5. **Seguridad:** Ocultar API keys del frontend

3.3.2 Estructura del Backend

server/

```
|— controllers/
|   |— weatherController.js
|   |— airQualityController.js
|   |— eventsController.js
|
|— routes/
|   |— weather.js
|   |— airQuality.js
|   |— events.js
|
|— services/
|   |— cacheService.js
|   |— apiService.js
|
|— middleware/
|   |— errorHandler.js
|   |— rateLimiter.js
|
|— config/
|   |— database.js
|
|— server.js
|— .env
```

3.4 Flujo de Datos

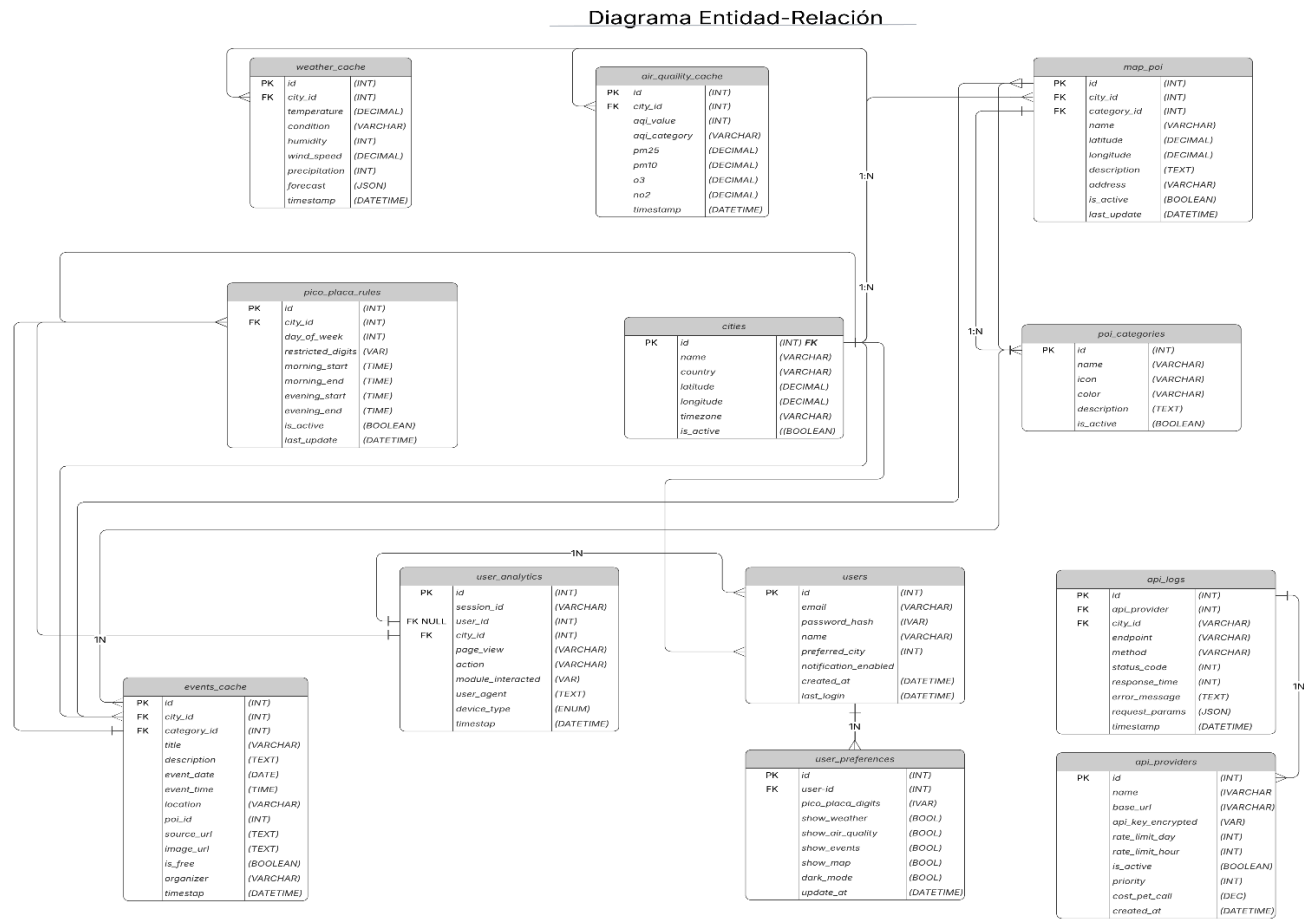


3.5 Estrategia de Caché

Tipo de Dato	TTL (Time To Live)	Estrategia
Clima	1 hora	Cache-aside
Calidad Aire	1 hora	Cache-aside
Eventos	24 horas	Cache-aside
Pico y Placa	24 horas	Cache-aside
Puntos Mapa	7 días	Cache-aside

4. DISEÑO DE BASE DE DATOS

4.1 Diagrama Entidad-Relación



4.1.1 Descripción de Relaciones

Relación: cities → weather_cache (1:N)

- **Tipo:** Uno a Muchos
- **Descripción:** Una ciudad puede tener múltiples registros históricos de clima
- **Cardinalidad:** (1,1) : (0,N)
- **Clave Foránea:** `weather\_cache.city\_id` → `cities.id`
- **Integridad:** ON DELETE CASCADE (si se elimina una ciudad, se eliminan sus registros)

Relación: cities → air_quality_cache (1:N)

- **Tipo:** Uno a Muchos
- **Descripción:** Una ciudad puede tener múltiples registros históricos de calidad del aire
- **Cardinalidad:** (1,1) : (0,N)
- **Clave Foránea:** `air\_quality\_cache.city\_id` → `cities.id`
- **Integridad:** ON DELETE CASCADE

Relación: cities → map_poi (1:N)

- **Tipo:** Uno a Muchos
- **Descripción:** Una ciudad tiene múltiples puntos de interés
- **Cardinalidad:** (1,1) : (0,N)
- **Clave Foránea:** `map\_poi.city\_id` → `cities.id`
- **Integridad:** ON DELETE CASCADE

Relación: poi_categories → map_poi (1:N)

- **Tipo:** Uno a Muchos
- **Descripción:** Una categoría puede tener múltiples puntos de interés
- **Cardinalidad:** (1,1) : (1,N)
- **Clave Foránea:** `map\_poi.category\_id` → `poi\_categories.id`
- **Integridad:** ON DELETE RESTRICT (no permitir eliminar categoría si tiene POIs)

Relación: cities → events_cache (1:N)

- **Tipo:** Uno a Muchos
- **Descripción:** Una ciudad tiene múltiples eventos
- **Cardinalidad:** (1,1) : (0,N)
- **Clave Foránea:** `events\_cache.city\_id` → `cities.id`
- **Integridad:** ON DELETE CASCADE

Relación: poi_categories → events_cache (1:N)

- **Tipo:** Uno a Muchos
- **Descripción:** Una categoría puede clasificar múltiples eventos
- **Cardinalidad:** (1,1) : (0,N)
- **Clave Foránea:** `events\_cache.category\_id` → `poi\_categories.id`

- **Integridad:** ON DELETE SET NULL

Relación: map_poi → events_cache (1:N) - OPCIONAL

- **Tipo:** Uno a Muchos (Opcional)
- **Descripción:** Un POI puede albergar múltiples eventos, o un evento puede no estar vinculado a un POI
- **Cardinalidad:** (0,1) : (0,N)
- **Clave Foránea:** `events\_cache.poi\_id` → `map\_poi.id` (NULLABLE)
- **Integridad:** ON DELETE SET NULL

Relación: cities → pico_placa_rules (1:N)

- **Tipo:** Uno a Muchos
- **Descripción:** Una ciudad tiene reglas de Pico y Placa para cada día
- **Cardinalidad:** (1,1) : (7,7)
- **Clave Foránea:** `pico\_placa\_rules.city\_id` → `cities.id`
- **Integridad:** ON DELETE CASCADE

Relación: cities → user_analytics (1:N)

- **Tipo:** Uno a Muchos
- **Descripción:** Los eventos de analytics están asociados a una ciudad
- **Cardinalidad:** (1,1) : (0,N)
- **Clave Foránea:** `user\_analytics.city\_id` → `cities.id`
- **Integridad:** ON DELETE SET NULL

Relación: users → user_analytics (1:N) - OPCIONAL

- **Tipo:** Uno a Muchos (Opcional - Fase 2)
- **Descripción:** Un usuario registrado puede tener múltiples eventos de analytics
- **Cardinalidad:** (0,1) : (0,N)
- **Clave Foránea:** `user\_analytics.user\_id` → `users.id` (NULLABLE)
- **Integridad:** ON DELETE SET NULL

Relación: users → user_preferences (1:1)

- **Tipo:** Uno a Uno (Fase 2)
- **Descripción:** Cada usuario tiene exactamente un conjunto de preferencias
- **Cardinalidad:** (1,1) : (1,1)
- **Clave Foránea:** `user\_preferences.user\_id` → `users.id` (UNIQUE)
- **Integridad:** ON DELETE CASCADE

Relación: cities → api_logs (1:N)

- **Tipo:** Uno a Muchos
- **Descripción:** Las llamadas a APIs están contextualizadas por ciudad
- **Cardinalidad:** (1,1) : (0,N)
- **Clave Foránea:** `api\_logs.city\_id` → `cities.id`
- **Integridad:** ON DELETE SET NULL (preservar logs históricos)

Relación: api_providers → api_logs (1:N)

-Tipo: Uno a Muchos

- Descripción: Un proveedor de API tiene múltiples llamadas registradas

- Cardinalidad: (1,1) : (0,N)

- Clave Foránea: `api\_logs.api\_provider\_id` → `api\_providers.id`

-Integridad: ON DELETE CASCADE (si se elimina proveedor, eliminar sus logs)

4.1.2 Tablas de Referencia (Maestras)

Tabla: cities

Propósito: Normalización para soportar múltiples ciudades en el futuro

sql

```
INSERT INTO cities (name, country, latitude, longitude, timezone, is_active) VALUES
('Bogotá', 'Colombia', 4.7110, -74.0721, 'America/Bogota', TRUE),
('Medellín', 'Colombia', 6.2442, -75.5812, 'America/Bogota', FALSE), -- Futuro
('Cali', 'Colombia', 3.4516, -76.5320, 'America/Bogota', FALSE); -- Futuro
```

Tabla: api_providers

Propósito: Centralizar configuración de proveedores de APIs y soportar múltiples alternativas

sql

```
INSERT INTO api_providers (name, base_url, rate_limit_day, rate_limit_hour, priority,
is_active) VALUES
```

-- Clima

```
('OpenWeatherMap', 'https://api.openweathermap.org/data/2.5/', 1000, 60, 1, TRUE),
('WeatherAPI', 'https://api.weatherapi.com/v1/', 1000000, 50000, 2, FALSE), --
Alternativa
```

-- Calidad del Aire

```
('AQICN', 'https://api.waqi.info/', 1000, 60, 1, TRUE),
('IQAIR', 'https://api.airvisual.com/v2/', 10000, 500, 2, FALSE), -- Alternativa
```

-- Eventos

```
('Eventbrite', 'https://www.eventbriteapi.com/v3/', 1000, 60, 1, TRUE),
```

-- Mapas

('OpenStreetMap', 'https://tile.openstreetmap.org/', 999999, 999999, 1, TRUE),
('Mapbox', 'https://api.mapbox.com/', 50000, 5000, 2, FALSE); -- Alternativa

4.2 Script SQL de Creación

Sql

BASE DE DATOS: CivicView

```
CREATE DATABASE IF NOT EXISTS civicview_db
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE civicview_db;
```

TABLAS MAESTRAS (Sin dependencias)

-- Tabla: Ciudades

```
CREATE TABLE cities (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL UNIQUE,
  country VARCHAR(100) NOT NULL DEFAULT 'Colombia',
  latitude DECIMAL(10, 8) NOT NULL,
  longitude DECIMAL(11, 8) NOT NULL,
  timezone VARCHAR(50) NOT NULL DEFAULT 'America/Bogota',
  is_active BOOLEAN DEFAULT TRUE,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_active (is_active),
  INDEX idx_coordinates (latitude, longitude)
) ENGINE=InnoDB;
```

-- Tabla: Categorías de POIs y Eventos

```
CREATE TABLE poi_categories (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL UNIQUE,
  icon VARCHAR(10) DEFAULT '📍',
  color VARCHAR(7) DEFAULT '#6B7280',
  description TEXT,
  is_active BOOLEAN DEFAULT TRUE,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_active (is_active)
) ENGINE=InnoDB;
```


-- Tabla: Proveedores de APIs

```
CREATE TABLE api_providers (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL UNIQUE,  
  service_type ENUM('weather', 'air_quality', 'events', 'maps') NOT NULL,  
  base_url VARCHAR(255) NOT NULL,  
  api_key_encrypted VARCHAR(255) COMMENT 'Clave encriptada',  
  rate_limit_day INT DEFAULT 1000 COMMENT 'Llamadas por día',  
  rate_limit_hour INT DEFAULT 60 COMMENT 'Llamadas por hora',  
  current_usage_day INT DEFAULT 0,  
  current_usage_hour INT DEFAULT 0,  
  is_active BOOLEAN DEFAULT TRUE,  
  priority INT DEFAULT 1 COMMENT '1=Principal, 2=Alternativa',  
  cost_per_call DECIMAL(10,4) DEFAULT 0.00 COMMENT 'Costo en USD',  
  last_reset DATETIME DEFAULT CURRENT_TIMESTAMP,  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  INDEX idx_active_priority (is_active, priority),  
  INDEX idx_service_type (service_type)  
) ENGINE=InnoDB;
```

-- =====

-- TABLAS CON RELACIONES

-- =====

-- Tabla: Caché de Clima

```
CREATE TABLE weather_cache (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  city_id INT NOT NULL,  
  temperature DECIMAL(5,2) NOT NULL,  
  feels_like DECIMAL(5,2),  
  condition VARCHAR(50) NOT NULL,  
  condition_code VARCHAR(10),  
  humidity INT,  
  wind_speed DECIMAL(5,2),  
  precipitation_prob INT,  
  forecast JSON COMMENT 'Array de pronóstico 7 días',  
  timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
```

-- Relaciones

```
FOREIGN KEY (city_id) REFERENCES cities(id)
```

```
ON DELETE CASCADE
ON UPDATE CASCADE,
```

-- Índices

```
INDEX idx_city_timestamp (city_id, timestamp),
INDEX idx_timestamp (timestamp)
) ENGINE=InnoDB;
```

-- Tabla: Caché de Calidad del Aire

```
CREATE TABLE air_quality_cache (
  id INT AUTO_INCREMENT PRIMARY KEY,
  city_id INT NOT NULL,
  aqi_value INT NOT NULL,
  aqi_category VARCHAR(50) NOT NULL,
  pm25 DECIMAL(6,2),
  pm10 DECIMAL(6,2),
  o3 DECIMAL(6,2),
  no2 DECIMAL(6,2),
  so2 DECIMAL(6,2),
  co DECIMAL(6,2),
  timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
```

-- Relaciones

```
FOREIGN KEY (city_id) REFERENCES cities(id)
ON DELETE CASCADE
ON UPDATE CASCADE,
```

-- Índices

```
INDEX idx_city_timestamp (city_id, timestamp),
INDEX idx_aqi_value (aqi_value)
) ENGINE=InnoDB;
```

-- Tabla: Puntos de Interés (POI)

```
CREATE TABLE map_poi (
  id INT AUTO_INCREMENT PRIMARY KEY,
  city_id INT NOT NULL,
  category_id INT NOT NULL,
  name VARCHAR(255) NOT NULL,
  latitude DECIMAL(10, 8) NOT NULL,
  longitude DECIMAL(11, 8) NOT NULL,
  description TEXT,
```

```
address VARCHAR(255),
phone VARCHAR(20),
website VARCHAR(255),
opening_hours JSON COMMENT 'Horarios de apertura',
is_active BOOLEAN DEFAULT TRUE,
last_update DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
```

-- Relaciones

```
FOREIGN KEY (city_id) REFERENCES cities(id)
ON DELETE CASCADE
ON UPDATE CASCADE,
FOREIGN KEY (category_id) REFERENCES poi_categories(id)
ON DELETE RESTRICT
ON UPDATE CASCADE,
```

-- Índices

```
INDEX idx_city_category (city_id, category_id),
INDEX idx_coordinates (latitude, longitude),
INDEX idx_active (is_active),
FULLTEXT idx_search (name, description)
) ENGINE=InnoDB;
```

-- Tabla: Caché de Eventos

```
CREATE TABLE events_cache (
id INT AUTO_INCREMENT PRIMARY KEY,
city_id INT NOT NULL,
category_id INT NOT NULL,
poi_id INT NULL COMMENT 'Referencia opcional al POI donde se realiza',
title VARCHAR(255) NOT NULL,
description TEXT,
event_date DATE NOT NULL,
event_time TIME,
end_date DATE,
end_time TIME,
location VARCHAR(255),
latitude DECIMAL(10, 8),
longitude DECIMAL(11, 8),
source_url TEXT,
image_url TEXT,
is_free BOOLEAN DEFAULT TRUE,
price_range VARCHAR(50),
organizer VARCHAR(255),
capacity INT,
```

```
registration_required BOOLEAN DEFAULT FALSE,  
timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
```

-- Relaciones

```
FOREIGN KEY (city_id) REFERENCES cities(id)  
ON DELETE CASCADE  
ON UPDATE CASCADE,  
FOREIGN KEY (category_id) REFERENCES poi_categories(id)  
ON DELETE SET NULL  
ON UPDATE CASCADE,  
FOREIGN KEY (poi_id) REFERENCES map_poi(id)  
ON DELETE SET NULL  
ON UPDATE CASCADE,
```

-- Índices

```
INDEX idx_city_date (city_id, event_date),  
INDEX idx_category (category_id),  
INDEX idx_date_range (event_date, end_date),  
INDEX idx_free_events (is_free),  
FULLTEXT idx_search (title, description)  
) ENGINE=InnoDB;
```

-- Tabla: Reglas de Pico y Placa

```
CREATE TABLE pico_placa_rules (  
id INT AUTO_INCREMENT PRIMARY KEY,  
city_id INT NOT NULL,  
day_of_week INT NOT NULL COMMENT '1=Lunes, 7=Domingo',  
restricted_digits VARCHAR(20) NOT NULL COMMENT 'Ej: 5-6',  
morning_start TIME DEFAULT '06:00:00',  
morning_end TIME DEFAULT '08:30:00',  
evening_start TIME DEFAULT '16:00:00',  
evening_end TIME DEFAULT '19:30:00',  
is_active BOOLEAN DEFAULT TRUE,  
effective_date DATE COMMENT 'Fecha desde cuando aplica',  
expiry_date DATE COMMENT 'Fecha hasta cuando aplica',  
last_update DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE  
CURRENT_TIMESTAMP,
```

-- Relaciones

```
FOREIGN KEY (city_id) REFERENCES cities(id)  
ON DELETE CASCADE  
ON UPDATE CASCADE,
```

-- Índices

```
    UNIQUE KEY unique_city_day (city_id, day_of_week),  
    INDEX idx_active (is_active)  
) ENGINE=InnoDB;
```

-- =====

-- TABLAS DE USUARIOS (FASE 2)

-- =====

-- Tabla: Usuarios

```
CREATE TABLE users (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    email VARCHAR(255) NOT NULL UNIQUE,  
    password_hash VARCHAR(255) NOT NULL,  
    name VARCHAR(255),  
    preferred_city INT,  
    notification_enabled BOOLEAN DEFAULT FALSE,  
    email_verified BOOLEAN DEFAULT FALSE,  
    is_active BOOLEAN DEFAULT TRUE,  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
    last_login DATETIME,
```

-- Relaciones

```
    FOREIGN KEY (preferred_city) REFERENCES cities(id)  
    ON DELETE SET NULL  
    ON UPDATE CASCADE,
```

-- Índices

```
    INDEX idx_email (email),  
    INDEX idx_active (is_active)  
) ENGINE=InnoDB;
```

-- Tabla: Preferencias de Usuario

```
CREATE TABLE user_preferences (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT NOT NULL UNIQUE,  
    pico_placa_digits VARCHAR(10) COMMENT 'Dígitos del vehículo',  
    show_weather BOOLEAN DEFAULT TRUE,  
    show_air_quality BOOLEAN DEFAULT TRUE,  
    show_events BOOLEAN DEFAULT TRUE,
```

```
show_map BOOLEAN DEFAULT TRUE,  
dark_mode BOOLEAN DEFAULT FALSE,  
language VARCHAR(5) DEFAULT 'es',  
updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE  
CURRENT_TIMESTAMP,
```

-- Relaciones

```
FOREIGN KEY (user_id) REFERENCES users(id)  
ON DELETE CASCADE  
ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

-- =====

-- TABLAS DE ANÁLISIS Y LOGS

-- =====

-- Tabla: Analíticas de Usuario

```
CREATE TABLE user_analytics (  
id INT AUTO_INCREMENT PRIMARY KEY,  
session_id VARCHAR(100) NOT NULL,  
user_id INT NULL COMMENT 'NULL para usuarios anónimos',  
city_id INT NOT NULL,  
page_view VARCHAR(100),  
action VARCHAR(50) COMMENT 'click, view, share, etc.',  
module_interacted VARCHAR(50) COMMENT 'pico_placa, weather, map, etc.',  
user_agent TEXT,  
device_type ENUM('mobile', 'tablet', 'desktop') DEFAULT 'mobile',  
ip_address VARCHAR(45),  
referrer VARCHAR(255),  
timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
```

-- Relaciones

```
FOREIGN KEY (user_id) REFERENCES users(id)  
ON DELETE SET NULL  
ON UPDATE CASCADE,  
FOREIGN KEY (city_id) REFERENCES cities(id)  
ON DELETE SET NULL  
ON UPDATE CASCADE,
```

-- Índices

```

    INDEX idx_session (session_id),
    INDEX idx_user_city (user_id, city_id),
    INDEX idx_timestamp (timestamp),
    INDEX idx_action (action)
) ENGINE=InnoDB;

```

-- Tabla: Logs de APIs

```

CREATE TABLE api_logs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    api_provider_id INT NOT NULL,
    city_id INT NULL COMMENT 'Ciudad contexto de la llamada',
    endpoint VARCHAR(255) NOT NULL,
    method VARCHAR(10) DEFAULT 'GET',
    status_code INT,
    response_time INT COMMENT 'Milisegundos',
    cache_hit BOOLEAN DEFAULT FALSE COMMENT 'Si se sirvió desde caché',
    error_message TEXT,
    request_params JSON,
    response_size INT COMMENT 'Bytes',
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,

```

-- Relaciones

```

    FOREIGN KEY (api_provider_id) REFERENCES api_providers(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (city_id) REFERENCES cities(id)
        ON DELETE SET NULL
        ON UPDATE CASCADE,

```

-- Índices

```

    INDEX idx_provider_status (api_provider_id, status_code),
    INDEX idx_provider_timestamp (api_provider_id, timestamp),
    INDEX idx_timestamp (timestamp),
    INDEX idx_cache_hit (cache_hit)
) ENGINE=InnoDB;

```

```
-- =====
```

-- DATOS INICIALES

```
-- =====
```

-- Insertar ciudad por defecto

```
INSERT INTO cities (name, country, latitude, longitude, timezone, is_active) VALUES ('Bogotá', 'Colombia', 4.7110, -74.0721, 'America/Bogota', TRUE);
```

-- Insertar categorías

```
INSERT INTO poi_categories (name, icon, color, description, is_active) VALUES ('Ciclovía', '🚲', '#10B981', 'Rutas para ciclistas', TRUE), ('Parque', '🌳', '#22C55E', 'Espacios verdes y recreativos', TRUE), ('Biblioteca', '📖', '#3B82F6', 'Bibliotecas públicas', TRUE), ('Centro Cultural', '🎭', '#8B5CF6', 'Centros culturales y teatros', TRUE), ('Museo', '🏛️', '#F59E0B', 'Museos y galerías', TRUE), ('Concierto', '🎵', '#EC4899', 'Eventos musicales', TRUE), ('Festival', '🎉', '#F97316', 'Festivales y celebraciones', TRUE), ('Deporte', '⚽', '#14B8A6', 'Eventos deportivos', TRUE), ('Cine', '🎬', '#6366F1', 'Proyecciones y cine', TRUE), ('Teatro', '🎪', '#A855F7', 'Obras de teatro', TRUE), ('Otro', '📍', '#6B7280', 'Sin categoría específica', TRUE);
```

-- Insertar proveedores de APIs

```
INSERT INTO api_providers (name, service_type, base_url, rate_limit_day, rate_limit_hour, priority, is_active) VALUES
```

-- Clima

```
('OpenWeatherMap', 'weather', 'https://api.openweathermap.org/data/2.5/', 1000, 60, 1, TRUE),
```

```
('WeatherAPI', 'weather', 'https://api.weatherapi.com/v1/', 1000000, 50000, 2, FALSE),
```

-- Calidad del Aire

```
('AQICN', 'air_quality', 'https://api.waqi.info/', 1000, 60, 1, TRUE),
```

```
('IQAIR', 'air_quality', 'https://api.airvisual.com/v2/', 10000, 500, 2, FALSE),
```

-- Eventos

```
('Eventbrite', 'events', 'https://www.eventbriteapi.com/v3/', 1000, 60, 1, TRUE),
```

-- Mapas

```
('OpenStreetMap', 'maps', 'https://tile.openstreetmap.org/', 999999, 999999, 1, TRUE),
```

```
('Mapbox', 'maps', 'https://api.mapbox.com/', 50000, 5000, 2, FALSE);
```


-- Insertar reglas de Pico y Placa para Bogotá (2025)

```
INSERT INTO pico_placa_rules (city_id, day_of_week, restricted_digits, is_active)
VALUES
```

```
(1, 1, '5-6', TRUE), -- Lunes
(1, 2, '7-8', TRUE), -- Martes
(1, 3, '9-0', TRUE), -- Miércoles
(1, 4, '1-2', TRUE), -- Jueves
(1, 5, '3-4', TRUE), -- Viernes
(1, 6, NULL, FALSE), -- Sábado
(1, 7, NULL, FALSE); -- Domingo
```

```
-- =====
```

-- PROCEDIMIENTOS ALMACENADOS ÚTILES

```
-- =====
```

DELIMITER //

-- Procedimiento: Limpiar caché antiguo

```
CREATE PROCEDURE clean_old_cache()
```

```
BEGIN
```

```
    DELETE FROM weather_cache WHERE timestamp < DATE_SUB(NOW(), INTERVAL 7
DAY);
```

```
    DELETE FROM air_quality_cache WHERE timestamp < DATE_SUB(NOW(),
INTERVAL 7 DAY);
```

```
    DELETE FROM events_cache WHERE end_date < DATE_SUB(NOW(), INTERVAL 30
DAY);
```

```
    DELETE FROM user_analytics WHERE timestamp < DATE_SUB(NOW(), INTERVAL
90 DAY);
```

```
    DELETE FROM api_logs WHERE timestamp < DATE_SUB(NOW(), INTERVAL 30
DAY);
```

```
END //
```

-- Procedimiento: Registrar llamada a API

```
CREATE PROCEDURE log_api_call(
```

```
    IN p_provider_id INT,
```

```
    IN p_city_id INT,
```

```
    IN p_endpoint VARCHAR(255),
```

```
    IN p_method VARCHAR(10),
```

```
    IN p_status_code INT,
```

```
    IN p_response_time INT,  
    IN p_cache_hit BOOLEAN,  
    IN p_error_message TEXT  
)  
BEGIN
```

```
    -- Insertar log  
    INSERT INTO api_logs (  
        api_provider_id,  
        city_id,  
        endpoint,  
        method,  
        status_code,  
        response_time,  
        cache_hit,  
        error_message  
    ) VALUES (  
        p_provider_id,  
        p_city_id,  
        p_endpoint,  
        p_method,  
        p_status_code,  
        p_response_time,  
        p_cache_hit,  
        p_error_message  
    );
```

-- Actualizar contador de uso si no es caché

```
    IF NOT p_cache_hit THEN  
        UPDATE api_providers  
        SET current_usage_day = current_usage_day + 1,  
            current_usage_hour = current_usage_hour + 1  
        WHERE id = p_provider_id;  
    END IF;  
END //
```

-- Función: Verificar si se puede usar API (rate limit)

```
CREATE FUNCTION can_use_api(p_provider_id INT)  
RETURNS BOOLEAN  
READS SQL DATA  
BEGIN  
    DECLARE v_can_use BOOLEAN DEFAULT FALSE;  
    DECLARE v_usage_day INT;  
    DECLARE v_usage_hour INT;  
    DECLARE v_limit_day INT;  
    DECLARE v_limit_hour INT;
```

```

SELECT
    current_usage_day,
    current_usage_hour,
    rate_limit_day,
    rate_limit_hour
INTO
    v_usage_day,
    v_usage_hour,
    v_limit_day,
    v_limit_hour
FROM api_providers
WHERE id = p_provider_id AND is_active = TRUE;

IF v_usage_day < v_limit_day AND v_usage_hour < v_limit_hour THEN
    SET v_can_use = TRUE;
END IF;

RETURN v_can_use;
END //

```

-- Función: Obtener API alternativa

```

CREATE FUNCTION get_alternative_api(p_service_type VARCHAR(50))
RETURNS INT
READS SQL DATA
BEGIN
    DECLARE v_provider_id INT;

    SELECT id INTO v_provider_id
    FROM api_providers
    WHERE service_type = p_service_type
        AND is_active = TRUE
        AND current_usage_day < rate_limit_day
        AND current_usage_hour < rate_limit_hour
    ORDER BY priority ASC
    LIMIT 1;

    RETURN v_provider_id;
END //

```

-- Función: Obtener regla de Pico y Placa para hoy

```

CREATE FUNCTION get_pico_placa_today(p_city_id INT)
RETURNS VARCHAR(20)
READS SQL DATA

```

```

BEGIN
  DECLARE v_digits VARCHAR(20);
  SELECT restricted_digits INTO v_digits
  FROM pico_placa_rules
  WHERE city_id = p_city_id
     AND day_of_week = DAYOFWEEK(NOW())
     AND is_active = TRUE
  LIMIT 1;
  RETURN v_digits;
END //

```

-- Función: Calcular categoría AQI

```

CREATE FUNCTION get_aqi_category(p_aqi INT)
RETURNS VARCHAR(50)
DETERMINISTIC
BEGIN
  IF p_aqi <= 50 THEN RETURN 'Bueno';
  ELSEIF p_aqi <= 100 THEN RETURN 'Moderado';
  ELSEIF p_aqi <= 150 THEN RETURN 'Dañino para grupos sensibles';
  ELSEIF p_aqi <= 200 THEN RETURN 'Dañino';
  ELSEIF p_aqi <= 300 THEN RETURN 'Muy dañino';
  ELSE RETURN 'Peligroso';
  END IF;
END //

```

DELIMITER ;

-- =====

-- VISTAS ÚTILES

-- =====

-- Vista: Eventos próximos

```

CREATE VIEW upcoming_events AS
SELECT
  e.id,
  e.title,
  e.description,
  e.event_date,
  e.event_time,
  e.location,
  c.name AS category,

```

```

    c.icon,
    city.name AS city_name,
    e.is_free,
    e.image_url,
    COALESCE(p.name, e.location) AS venue_name,
    p.latitude,
    p.longitude
FROM events_cache e
INNER JOIN cities city ON e.city_id = city.id
INNER JOIN poi_categories c ON e.category_id = c.id
LEFT JOIN map_poi p ON e.poi_id = p.id
WHERE e.event_date >= CURDATE()
    AND city.is_active = TRUE
ORDER BY e.event_date ASC, e.event_time ASC;

```

-- Vista: Estadísticas de APIs

```

CREATE VIEW api_statistics AS
SELECT
    p.name AS provider_name,
    p.service_type,
    COUNT(l.id) AS total_calls,
    SUM(CASE WHEN l.cache_hit = TRUE THEN 1 ELSE 0 END) AS cache_hits,
    SUM(CASE WHEN l.status_code >= 200 AND l.status_code < 300 THEN 1 ELSE 0
END) AS successful_calls,
    SUM(CASE WHEN l.status_code >= 400 THEN 1 ELSE 0 END) AS failed_calls,
    AVG(l.response_time) AS avg_response_time,
    MAX(l.response_time) AS max_response_time,
    MIN(l.response_time) AS min_response_time,
    (SUM(CASE WHEN l.cache_hit = TRUE THEN 1 ELSE 0 END) / COUNT(l.id) * 100) AS
cache_hit_rate,
    DATE(l.timestamp) AS date
FROM api_providers p
LEFT JOIN api_logs l ON p.id = l.api_provider_id
WHERE l.timestamp >= DATE_SUB(NOW(), INTERVAL 30 DAY)
GROUP BY p.id, DATE(l.timestamp)
ORDER BY date DESC, provider_name ASC;

```

-- Vista: Rate limit status

```

CREATE VIEW api_rate_limit_status AS
SELECT
    id,
    name,
    service_type,
    current_usage_day,
    rate_limit_day,

```

```

(current_usage_day / rate_limit_day * 100) AS usage_percentage_day,
current_usage_hour,
rate_limit_hour,
(current_usage_hour / rate_limit_hour * 100) AS usage_percentage_hour,
is_active,
priority,
last_reset
FROM api_providers
WHERE is_active = TRUE
ORDER BY service_type, priority;

```

4.3 Políticas de Retención de Datos

Tabla	Retención	Razón
weather_cache	7 días	Datos históricos no son prioritarios
air_quality_cache	7 días	Similar a clima
events_cache	30 días	Útil mantener eventos pasados por analítica
user_analytics	90 días	Compliance y análisis de tendencias
map_poi	Indefinido	Datos maestros
pico_placa_rules	Indefinido	Datos de configuración

4.4 Estrategia de Backup

- Frecuencia: Backup diario automático a las 02:00 AM
- Retención: 7 backups diarios + 4 backups mensuales
- Herramienta: mysqldump o servicio nativo del hosting

5. INVESTIGACIÓN Y ANÁLISIS DE APIS

5.1 API de Clima: OpenWeatherMap

URL Base: `https://api.openweathermap.org/data/2.5/`

Endpoints Utilizados

1. Current Weather Data
GET /weather?q=Bogota&appid={API_KEY}&units=metric&lang=es
2. 5 Day Forecast
GET /forecast?q=Bogota&appid={API_KEY}&units=metric&lang=es

Límites del Plan Gratuito

- Llamadas: 1,000 por día (~42 por hora)
- Datos incluidos: Clima actual, pronóstico 5 días (intervalos 3h)
- Actualización: Cada 10 minutos desde la fuente

Estructura de Respuesta (Current Weather)

```
json
{
  "main": {
    "temp": 18.5,
    "feels_like": 17.2,
    "humidity": 72
  },
  "weather": [{
    "main": "Clouds",
    "description": "nubes dispersas",
    "icon": "02d"
  }],
  "wind": {
    "speed": 3.5
  },
  "dt": 1697040000
}
```

Estrategia de Uso

- Caché: 1 hora
- Fallback: Mostrar último dato conocido + advertencia
- Alternativa: WeatherAPI.com (5M calls/mes gratis)

5.2 API de Calidad del Aire: AQICN

URL Base: `https://api.waqi.info/`

Endpoints Utilizados

GET /feed/bogota/?token={API_KEY}

Límites del Plan Gratuito

- Llamadas: 1,000 por día
- Datos incluidos: AQI, PM2.5, PM10, O3, NO2, SO2
- Actualización: Cada hora desde estaciones gubernamentales

Estructura de Respuesta

```
json
{
  "status": "ok",
  "data": {
    "aqi": 65,
    "idx": 243,
    "city": {
      "name": "Bogota"
    },
    "iaqi": {
      "pm25": {"v": 42},
      "pm10": {"v": 58}
    },
    "time": {
      "s": "2025-10-15 14:00:00"
    }
  }
}
```

Categorización del AQI

Rango AQI	Categoría	Color	Recomendación
0-50	Bueno	Verde	Calidad del aire satisfactoria
51-100	Moderado	Amarillo	Aceptable para la mayoría
101-150	Dañino para grupos sensibles	Naranja	Limitar actividades prolongadas al aire libre
151-200	Dañino	Rojo	Evitar esfuerzos prolongados al aire libre
201-300	Muy dañino	Morado	Permanecer en interiores
301+	Peligroso	Marrón	Evitar todas las actividades al aire libre

Estrategia de Uso

- Caché: 1 hora
- Fallback: IQAir API (alternativa)
- Visualización: Tarjeta con código de color + recomendaciones

5.3 API de Eventos: Eventbrite

URL Base: `https://www.eventbriteapi.com/v3/`

Endpoints Utilizados

GET /events/search/?location.address=Bogota&location.within=10km&token={API_KEY}

Límites del Plan Gratuito

- Llamadas: 1,000 por día
- Filtros: Ubicación, fecha, categoría, precio
- Datos: Título, descripción, fecha, lugar, imagen

Estructura de Respuesta

```
json
{
  "events": [{
    "name": {
      "text": "Concierto en el Parque Simón Bolívar"
    },
    "description": {
      "text": "Evento musical gratuito..."
    },
    "start": {
      "local": "2025-10-20T18:00:00"
    },
    "venue": {
      "name": "Parque Simón Bolívar",
      "address": {
        "city": "Bogotá"
      }
    },
    "is_free": true,
    "logo": {
      "url": "https://..."
    }
  }]
}
```

Alternativas

- Cultura Recreación y Deporte Bogotá: API oficial (si existe)
- Web Scraping: Como último recurso (ético y con rate limiting)

Estrategia de Uso

- Caché: 24 horas
- Filtro: Solo eventos gratuitos inicialmente
- Actualización: Diaria a las 00:00

5.4 Servicio de Mapas: Leaflet + OpenStreetMap

Librería: Leaflet.js (open source)

Tiles: OpenStreetMap (gratuito, ilimitado)

Configuración Básica

```
javascript
```

```
const map = L.map('map').setView([4.6097, -74.0817], 11);
```

```
L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {  
  attribution: '© OpenStreetMap contributors',  
  maxZoom: 18  
}).addTo(map);
```

Datos GeoJSON

Fuente: IDECA (Infraestructura de Datos Espaciales para el Distrito Capital)

Capas a incluir:

1. Ciclovías de Bogotá
2. Parques públicos
3. Bibliotecas
4. Centros culturales

Estrategia

- Carga inicial: Cargar GeoJSON completo al inicio
- Optimización: Clustering de marcadores cuando hay +100 puntos
- Caché: LocalStorage del navegador (7 días)

5.5 Resumen Comparativo de APIs

API	Proveedor	Costo	Límite Diario	Latencia	Confiabilidad	Alternativa
Clima	OpenWeatherMap	Gratis	1,000	~200ms	99.9%	WeatherAPI
Calidad Aire	AQICN	Gratis	1,000	~300ms	98%	IQAir
Eventos	Eventbrite	Gratis	1,000	~400ms	95%	Web scraping
Mapas	Leaflet+OSM	Gratis	Ilimitado	~100ms	99.9%	Google Maps

6. DISEÑO DE LA EXPERIENCIA DE USUARIO (UX)

6.1 Investigación de Usuarios

6.1.1 Perfil de Usuario Primario

Nombre: Carlos Andrés

Edad: 28 años

Ocupación: Profesional en tecnología

Ubicación: Chapinero, Bogotá

Dispositivo principal: Smartphone (Android)

Comportamiento:

- Revisa su celular al despertar (6:30 AM)
- Consulta clima antes de salir
- Usa carro ocasionalmente (preocupado por Pico y Placa)
- Busca eventos culturales los fines de semana

Frustraciones:

- "Tengo que abrir 5 apps diferentes cada mañana"
- "Siempre olvido revisar el Pico y Placa"
- "No sé qué eventos culturales hay cerca de mí"

Objetivos:

- Planificar su día en menos de 2 minutos
- Tomar decisiones informadas sobre movilidad
- Descubrir actividades culturales gratuitas

6.1.2 User Journey Map

FASE 1: Despertar (6:30 AM)

Acción: Revisa el celular en la cama

Pensamiento: "¿Qué necesito saber para hoy?"

Emoción: 😐 Neutro

Touchpoint: Abre CivicView desde bookmark

↓

FASE 2: Consulta Rápida (6:31 AM)

Acción: Escanea dashboard en 10 segundos

Pensamiento: "Ok, no puedo sacar el carro, está nublado, el aire está bien"

Emoción: 😊 Satisfecho

Touchpoint: Vista del dashboard principal



FASE 3: Decisión (6:32 AM)

Acción: Decide ir en TransMilenio y llevar chaqueta

Pensamiento: "Perfecto, ya sé qué hacer"

Emoción: 😊 Tranquilo

Touchpoint: Cierra la app



FASE 4: Exploración Opcional (Tarde)

Acción: Abre CivicView desde el trabajo

Pensamiento: "¿Hay algo interesante para hacer hoy?"

Emoción: 😊 Curioso

Touchpoint: Revisa sección de eventos y mapa



FASE 5: Acción (Después del trabajo)

Acción: Va a un evento cultural gratis que encontró

Pensamiento: "Qué bueno que esta app me avisó"

Emoción: 😊 Feliz

Touchpoint: Compartió CivicView con amigos

6.2 Arquitectura de Información

6.2.1 Mapa del Sitio

CivicView



└─ Home (Dashboard Principal)

- | | — Tarjeta: Pico y Placa
- | | — Tarjeta: Clima
- | | — Tarjeta: Calidad del Aire
- | | — Tarjeta: Eventos Hoy
- | | — Previsualización: Mapa
- |
- | — Mapa Completo (Vista Expandida)
- | | — Filtros por categoría
- | | — Marcadores de POI
- | | — Información de puntos
- |
- | — Configuración (Fase 2)
 - | — Preferencias de notificaciones
 - | — Dígitos de Pico y Placa guardados

6.2.2 Jerarquía Visual (Prioridad de Información)

Nivel 1 (Crítico): Pico y Placa + Clima

Nivel 2 (Importante): Calidad del Aire

Nivel 3 (Útil): Eventos + Mapa

6.3 Wireframes de Baja Fidelidad

6.3.1 Dashboard Principal - Móvil (375px)

CIVICVIEW

INICIAR SESION

 HOY, 15 DIC 2025

PICO Y PLACA

 HOY SÍ PUEDES
CIRCULAR

RESTRINGIDO: 5-6
MAÑANA: 7-8 

CLIMA

18°C
 NUBL.
 60%

[MÁS →]

AIRE

MODERADO
ICA: 65
 ALERT

[INFO]





EVENTOS HOY (3)

• CONCIERTO PARQUE
SIMÓN BOLÍVAR
🕒 18:00 - GRATIS

• FESTIVAL DE JAZZ
ZONA T
🕒 20:00 - GRATIS



• SIMÓN BOLÍVAR
18:00 - GRATIS

• FESTIVAL DE JAZZ
ZONA T

20:00 - GRATIS



MAPA INTERACTIVO



FILTROS:

- ☒ CICLOVÍAS
- ☒ BIBLIOTECAS
- ☒ PARQUES
- ☒ CULTURA

6.3.2 Dashboard Principal - Desktop (1440px)

CIVICVIEW

INICIAR SESION

HOY, 15 DIC 2025

PICO Y PLACA

HOY SÍ PUEDES CIRCULAR

RESTRINGIDO: 5-6
MAÑANA: 7-8 





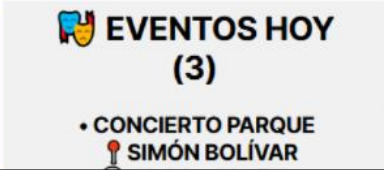
CLIMA

18°C

NUBL.
60%

[MÁS →]





AIRE

MODERADO

ICA: 65
ALERT

[INFO]





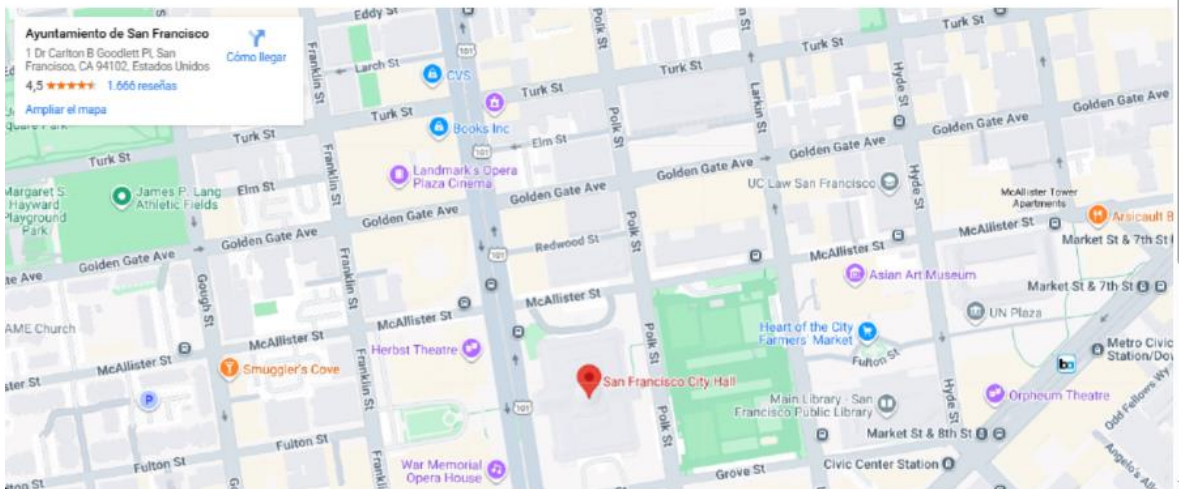


 **EVENTOS HOY**
(3)

- **CONCIERTO PARQUE**
 **SIMÓN BOLÍVAR**
 **18:00 - GRATIS**
- **FESTIVAL DE JAZZ**
 **ZONA T**
 **20:00 - GRATIS**



MAPA INTERACTIVO



6.4 Flujos de Usuario

Flujo 1: Consulta Matutina (Uso Principal)

1. Usuario abre bookmark/app
2. Dashboard carga en < 2 segundos
3. Usuario escanea visualmente (patrón F)
4. Identifica información crítica
5. Toma decisión
6. Cierra la app

Tiempo estimado: 30-60 segundos

Flujo 2: Búsqueda de Eventos

1. Usuario abre CivicView
2. Scroll hacia sección "Eventos"
3. Revisa lista de eventos hoy
4. Click en "Ver todos" (opcional)
5. Click en evento específico
6. Ve detalles: ubicación, hora
7. Abre mapa para ver ubicación
8. Decide asistir

Tiempo estimado: 2-3 minutos

Flujo 3: Exploración de Mapa

1. Usuario click en "Abrir Mapa Completo"
2. Vista mapa expandido en pantalla completa
3. Filtra por categoría (ej: solo parques)
4. Hace zoom en zona de interés
5. Click en marcador específico
6. Ve información del punto
7. Guarda ubicación (futuro) / comparte

Tiempo estimado: 3-5 minutos

6.5 Principios de Diseño UX

1. Claridad sobre Creatividad

- La información debe ser inmediatamente comprensible
- Evitar elementos decorativos que distraigan
- Priorizar legibilidad sobre estética

2. Diseño Mobile-First

- Desarrollar primero para móvil
- Progressive enhancement para desktop
- Gestos táctiles intuitivos (swipe, tap)

3. Jerarquía Visual Clara

- Usar tamaño, color y espaciado para guiar la atención
- Información crítica arriba y más grande
- Información secundaria con menor prominencia

4. Feedback Inmediato

- Loading states visibles
- Confirmación de acciones
- Mensajes de error claros y accionables

5. Consistencia

- Mismo estilo de tarjetas en toda la app
- Iconografía coherente
- Terminología uniforme

6. Accesibilidad (WCAG 2.1 AA)

- Contraste mínimo 4.5:1 para texto
- Tamaño de fuente legible (16px+)
- Etiquetas ARIA para lectores de pantalla
- Navegación por teclado

7. DISEÑO DE LA INTERFAZ DE USUARIO (UI)

7.1 Sistema de Diseño (Design System)

7.1.1 Paleta de Colores

Colores Primarios:

css

```
--primary-blue: #2C5F8D;    /* Azul Bogotá */
--primary-yellow: #F4B942;   /* Amarillo acento */
--primary-dark: #1A3A52;    /* Azul oscuro */
```

Colores Semánticos:

css

Estado de Calidad del Aire

```
--aqi-good: #10B981;        /* Verde - Bueno */
--aqi-moderate: #F59E0B;    /* Amarillo - Moderado */
--aqi-unhealthy-sensitive: #FB923C; /* Naranja */
--aqi-unhealthy: #EF4444;   /* Rojo - Dañino */
--aqi-very-unhealthy: #9333EA; /* Morado */
--aqi-hazardous: #7C2D12;   /* Marrón - Peligroso */
```

Pico y Placa

```
--pp-allowed: #10B981;      /* Verde - Puede circular */
--pp-restricted: #EF4444;   /* Rojo - No puede circular */
```

Sistema

```
--success: #10B981;  
--warning: #F59E0B;  
--error: #EF4444;  
--info: #3B82F6;
```

Colores Neutrales:

css

```
--gray-50: #F9FAFB;  
--gray-100: #F3F4F6;  
--gray-200: #E5E7EB;  
--gray-300: #D1D5DB;  
--gray-500: #6B7280;  
--gray-700: #374151;  
--gray-900: #111827;
```

7.1.2 Tipografía

Familia tipográfica: Inter (Google Fonts)

css

Títulos

```
--font-h1: 600 32px/40px 'Inter'; /* Hero */  
--font-h2: 600 24px/32px 'Inter'; /* Page title */  
--font-h3: 600 20px/28px 'Inter'; /* Card title */  
--font-h4: 600 18px/24px 'Inter'; /* Section title */
```

Cuerpo

```
--font-body-lg: 400 18px/28px 'Inter'; /* Body large */  
--font-body: 400 16px/24px 'Inter'; /* Body default */  
--font-body-sm: 400 14px/20px 'Inter'; /* Body small */
```

/* Datos/Métricas */

```
--font-metric: 700 48px/56px 'Inter'; /* Grande (temperatura) */  
--font-metric-md: 600 36px/44px 'Inter'; /* Mediana */  
--font-metric-sm: 600 24px/32px 'Inter'; /* Pequeña */
```

/* Auxiliar */

```
--font-caption: 400 12px/16px 'Inter'; /* Timestamps */
```

7.1.3 Espaciado (Sistema de 8px)

css

```
--space-1: 4px;  
--space-2: 8px;  
--space-3: 12px;  
--space-4: 16px;  
--space-5: 20px;  
--space-6: 24px;  
--space-8: 32px;  
--space-10: 40px;  
--space-12: 48px;  
--space-16: 64px;
```

Uso

- Padding interno de cards: `--space-6` (24px)
- Gap entre cards: `--space-4` (16px)
- Margin entre secciones: `--space-8` (32px)

7.1.4 Sombras (Elevación)

```
``css
```

```
--shadow-sm: 0 1px 2px rgba(0, 0, 0, 0.05);  
--shadow-md: 0 4px 6px rgba(0, 0, 0, 0.07);  
--shadow-lg: 0 10px 15px rgba(0, 0, 0, 0.1);  
--shadow-xl: 0 20px 25px rgba(0, 0, 0, 0.15);
```

Uso:

- Cards en reposo: `--shadow-md`
- Cards en hover: `--shadow-lg`
- Modales: `--shadow-xl`

7.1.5 Bordes y Radios

```
css
```

```
--radius-sm: 4px; /* Botones pequeños */  
--radius-md: 8px; /* Cards, inputs */  
--radius-lg: 12px; /* Cards destacadas */  
--radius-xl: 16px; /* Contenedores grandes */  
--radius-full: 9999px; /* Círculos */  
--border-width: 1px;  
--border-color: var(--gray-200);
```

7.1.6 Iconografía

Librería: Lucide Icons (React)

Tamaños estándar

```
css
```

--icon-sm: 16px;
--icon-md: 24px;
--icon-lg: 32px;
--icon-xl: 48px;

Iconos principales:

- 🚗 Pico y Placa: `Car`
- 🌧️ Clima: `CloudRain`, `Sun`, `Cloud`
- 🌀 Calidad Aire: `Wind`
- 📅 Eventos: `Calendar`
- 🗺️ Mapa: `Map`, `MapPin`
- 🔍 Buscar: `Search`
- ☰ Menú: `Menu`

7.2 Componentes UI

7.2.1 Card (Componente Base)

jsx

```
<Card variant="elevated">  
  <CardHeader icon={<Icon />} title="Título" />  
  <CardBody>  
    {/* Contenido */}  
  </CardBody>  
  <CardFooter timestamp="Hace 5 min" />  
</Card>
```

Variantes:

- `elevated`: Con sombra (defecto)
- `outlined`: Solo borde
- `flat`: Sin elevación

Estados:

- Default
- Hover (sombra más pronunciada)
- Loading (skeleton)
- Error (borde rojo)

7.2.2 Módulo Pico y Placa - Especificaciones

Tamaños:

- Móvil: Ancho completo, height: auto
- Desktop: 400px × 280px

Elementos:

1. Header:

- Icono: Car (24px)
- Título: "Pico y Placa" (font-h3)

2. Body:

- Estado visual grande:
 -  "HOY SÍ PUEDES CIRCULAR" (verde)
 -  "HOY NO PUEDES CIRCULAR" (rojo)
- Dígitos restringidos: "5 - 6" (font-metric-sm)
- Horarios: "6:00 AM - 8:30 AM / 4:00 PM - 7:30 PM"

3. Footer:

- Preview de mañana: "Mañana: 7-8 

Estados:

- Puede circular: Fondo verde claro, check verde
- No puede circular: Fondo rojo claro, X roja
- Fin de semana: Fondo gris, texto "No aplica"

7.2.3 Módulo Clima - Especificaciones

Tamaños:

- Móvil: 50% ancho (lado a lado con Calidad Aire)
- Desktop: 280px × 280px

Elementos:

1. Header:

- Icono dinámico según condición
- Título: "Clima"

2. Body:

- Temperatura: 18°C (font-metric)

- Condición: "Nublado" (font-body)
- Icono grande (icon-xl)
- Prob. lluvia: 💧 60%
- Humedad: 72%

3. Footer:

- "Actualizado hace 15 min"

Íconos por condición:

- Soleado: `Sun`
- Nublado: `Cloud`
- Lluvia: `CloudRain`
- Tormenta: `CloudLightning`

7.2.4 Módulo Calidad del Aire - Especificaciones

Tamaños:

- Móvil: 50% ancho
- Desktop: 280px × 280px

Elementos:

1. Header:

- Icono: `Wind`
- Título: "Calidad del Aire"

2. Body:

- Badge de color según ICA
- Categoría: "Moderado" (font-h3)
- Valor ICA: 65 (font-metric-md)
- Barra de progreso visual (0-300)
- Recomendación breve

3. Footer:

- Link: "Ver detalles →"

Colores dinámicos:

javascript

```
const getAQIColor = (aqi) => {
  if (aqi <= 50) return 'green';
  if (aqi <= 100) return 'yellow';
```

```

if (aqi <= 150) return 'orange';
if (aqi <= 200) return 'red';
if (aqi <= 300) return 'purple';
return 'brown';
}

```

7.2.5 Módulo Eventos - Especificaciones

Layout:

- Lista vertical de eventos
- Máximo 3 eventos visibles
- Botón "Ver todos" al final

Item de evento:

• Concierto en el Parque	
 Parque Simón Bolívar	
 18:00 • Gratis	

Elementos:

- Título (font-body, bold)
- Ubicación con icono MapPin
- Hora con icono Clock
- Badge "Gratis" (si aplica)

7.2.6 Módulo Mapa - Especificaciones

Vista Preview (Dashboard)

- Height: 300px (móvil), 400px (desktop)
- Mapa estático centrado en Bogotá

- 5-10 puntos destacados visibles
- Botón overlay: "Abrir Mapa Completo"

Vista Completa:

- Pantalla completa o modal
- Controles de zoom
- Sidebar con leyenda
- Filtros por categoría

Marcadores:

```
javascript
const markerIcons = {
  ciclovía: '🚲',
  parque: '🌳',
  biblioteca: '📖',
  cultura: '🎨'
}
```

7.3 Responsive Design

Breakpoints

```
css
/* Mobile First */
--screen-sm: 640px; /* Tablet pequeña */
--screen-md: 768px; /* Tablet */
--screen-lg: 1024px; /* Desktop */
--screen-xl: 1280px; /* Desktop grande */
```

Layout por Dispositivo

Móvil (< 640px):

- 1 columna
- Cards apiladas verticalmente
- Padding: 16px
- Font-size base: 16px

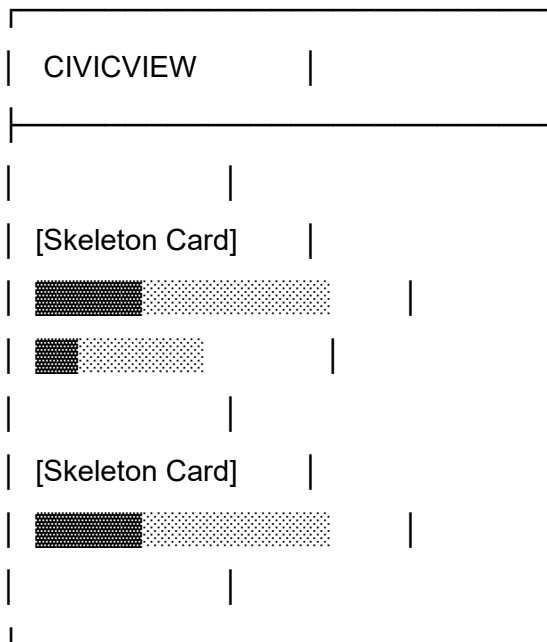
Tablet (640px - 1023px):

- 2 columnas (Grid 2x2)
- Cards más compactas
- Padding: 24px

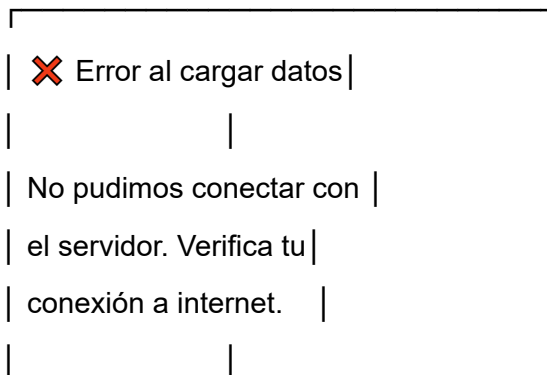
- Font-size base: 16px
- Desktop (1024px+):
- 3 columnas o layout híbrido
- Sidebar opcional
- Max-width: 1440px (centrado)
- Padding: 32px
- Font-size base: 16px

7.4 Estados de la Aplicación

Loading State



Error State



| [Reintentar] [Cancelar] |

|

Empty State

|

|  No hay eventos hoy |

|

| No encontramos eventos |

| culturales para hoy. |

| Vuelve mañana. |

|

|

8. STACK TECNOLÓGICO

8.1 Frontend

Framework Principal

React 18.3+

- Razón: Ecosistema maduro, gran comunidad, componentes reutilizables
- Ventajas: Virtual DOM, hooks, performance
- Alternativas evaluadas: Vue, Svelte

Herramientas de Build

Vite 5+

- Razón: Build ultrarrápido, HMR instantáneo
- Ventajas: Configuración mínima, optimización automática
- Alternativa: Create React App (deprecated)

Estilos

TailwindCSS 3+

- Razón: Utility-first, altamente personalizable, producción ligera
- Ventajas: No CSS custom, diseño consistente, responsive fácil
- Complemento: PostCSS

Estado Global

React Context + Hooks

- Razón: Suficiente para complejidad del proyecto
- Alternativa (si crece): Zustand, Redux Toolkit

Routing

React Router 6+

- Razón: Estándar de la industria
- **Uso:** Navegación entre Dashboard y Mapa completo

Manejo de Peticiones HTTP

Axios 1.6

- Razón: Interceptors, timeouts, mejor manejo de errores que fetch
- Configuración: Instancia centralizada con config base

Mapas

Leaflet 1.9+ + React-Leaflet

- **Razón:** Open source, sin límites de uso
- Tiles: OpenStreetMap
- Plugins: MarkerCluster, GeoJSON

Iconos

Lucide React

- Razón: Moderna, tree-shakeable, consistente
- Alternativas: React Icons, Heroicons

8.2 Backend (Opcional)

Runtime

Node.js 20 LTS + Express.js 4+******

- Razón: JavaScript full-stack, ecosistema rico
- Alternativa: Fastify (más rápido)

Base de Datos

MySQL 8

- Razón: Requisito del proyecto, confiable, bien documentado

- ORM: Sequelize o Prisma

Caché

node-cache (in-memory)

- Razón: Simple, suficiente para MVP
- Alternativa futura: Redis

Manejo de Variables de Entorno

dotenv*

env

OPENWEATHER_API_KEY=xxx

AQICN_API_KEY=xxx

EVENTBRITE_API_KEY=xxx

DB_HOST=localhost

DB_USER=root

DB_PASSWORD=xxx

DB_NAME=civicview_db

8.3 Herramientas de Desarrollo

Control de Versiones

Git + GitHub

- Ramas: `main`, `develop`, `feature/\*`
- Commits convencionales: `feat:`, `fix:`, `docs:`

Editor

VS Code

- Extensiones:
 - ESLint
 - Prettier
 - Tailwind CSS IntelliSense
 - ES7+ React Snippets

Testing (Fase 4)

- Jest: Unit testing
- React Testing Library: Component testing
- Cypress: E2E testing (opcional)

API Testing

Postman / Insomnia

- Colecciones guardadas para cada API
- Variables de entorno

8.4 Despliegue

Frontend Hosting

Vercel (opción preferida)

- Razón: Deploy automático desde Git, CDN global, SSL gratis
- Alternativa: Netlify, GitHub Pages

Backend Hosting

Railway (opción preferida)

- Razón: Free tier generoso, MySQL incluido, fácil setup
- Alternativas: Render, Heroku (no gratuito), Fly.io

Base de Datos

Railway MySQL o PlanetScale (free tier)

Dominio

Gratuito: Subdominio de Vercel (civicview.vercel.app)

Futuro: Dominio custom (civicview.co)

8.5 Justificación de Selección Tecnológica

Criterio	Peso	React	Vue	Angular	Decisión
Comunidad	20%	10	8	7	React
Curva de aprendizaje	15%	8	9	6	-
Performance	20%	9	9	8	-
Ecosistema	20%	10	7	8	React
Empleo/Portfolio	15%	10	7	8	React
Documentación	10%	9	8	9	-
TOTAL	100%	9.35	8.05	7.45	 React

9. PLAN DE IMPLEMENTACIÓN

9.1 Estrategia de Desarrollo

Metodología

Agile - Scrum adaptado

- Sprints de 2 semanas
- Daily standups personales (checklist diario)
- Retrospectivas al final de cada fase

Priorización: Método MoSCoW

Must Have (MVP):

- ☒ Módulo Pico y Placa
- ☒ Integración API Clima
- ☒ Integración API Calidad del Aire
- ☒ Dashboard responsive
- ☒ Despliegue en producción

Should Have:

- ☒ Módulo Eventos
- ☒ Mapa interactivo básico
- ☒ Caché de datos

Could Have:

- ☐ Filtros avanzados en mapa
- ☐ Gráficos de tendencias
- ☐ Modo oscuro

Won't Have (Fase 1):

- ☐ Notificaciones push
- ☐ Autenticación de usuarios
- ☐ Personalización de dashboard
- ☐ App móvil nativa

9.2 Estructura del Repositorio Git

civicview/

```
|— .github/
|   |— workflows/
|       |— deploy.yml      # CI/CD
|— frontend/
|   |— public/
|       |— favicon.ico
|       |— manifest.json   # PWA
|       |— robots.txt
|   |— src/
|       |— components/    # Como se especificó antes
|       |— services/
|       |— hooks/
|       |— utils/
|       |— context/
|       |— styles/
|       |— App.jsx
|       |— main.jsx
|   |— .env.example
|   |— .eslintrc.json
|   |— .prettierrc
|   |— package.json
|   |— tailwind.config.js
|   |— vite.config.js
|   |— README.md
|— backend/                # Opcional
|   |— controllers/
|   |— routes/
```

```
| |— services/
| |— middleware/
| |— config/
| |— server.js
| |— .env.example
| |— package.json
|— database/
| |— schema.sql
| |— migrations/
| |— seeds/
|— docs/
| |— arquitectura.md
| |— api-docs.md
| |— user-guide.md
|— .gitignore
|— README.md
|— LICENSE
```

9.3 Checklist de Desarrollo por Fase

✓ Fase 1: Planificación y Diseño (ACTUAL)

- [x] Definir requisitos funcionales y no funcionales
- [x] Investigar y documentar APIs
- [x] Diseñar arquitectura del sistema
- [x] Crear modelo de base de datos
- [x] Desarrollar wireframes
- [x] Crear diseño UI en Figma
- [x] Definir Design System
- [x] Prototipo interactivo
- [x] Documentación técnica completa

Entregable: Este documento + Prototipos Figma

Fase 2: Desarrollo Frontend (Próxima)

Semana 1-2:

- ☐ Configurar proyecto React + Vite
- ☐ Instalar dependencias (Tailwind, Axios, Router)
- ☐ Configurar ESLint + Prettier
- ☐ Crear estructura de carpetas
- ☐ Implementar Design System en CSS
- ☐ Crear componentes base (Card, Button, Loading)

Semana 3-4:

- ☐ Desarrollar componente Dashboard
- ☐ Implementar módulo Pico y Placa (lógica JS)
- ☐ Crear componente Header y Footer
- ☐ Implementar responsive design
- ☐ Testing manual en múltiples dispositivos

Entregable: Frontend estático funcional

Fase 3: Integración APIs y Backend

Semana 1:

- ☐ Registrar en todas las APIs
- ☐ Obtener API keys
- ☐ Crear servicios de API en frontend
- ☐ Implementar llamadas a OpenWeatherMap
- ☐ Implementar llamadas a AQICN

Semana 2:

- ☐ Implementar llamadas a Eventbrite
- ☐ Integrar Leaflet para mapa
- ☐ Cargar GeoJSON de puntos de interés
- ☐ Manejo de estados de carga y error

Semana 3-4:

- ☐ Setup backend Node.js + Express (si se usa)
 - ☐ Configurar MySQL
 - ☐ Crear endpoints proxy
 - ☐ Implementar sistema de caché
 - ☐ Conectar frontend con backend
- Entregable: Aplicación completamente integrada con datos reales

Fase 4: Pruebas y Despliegue

Semana 1-2:

- ☐ Pruebas unitarias de componentes críticos
- ☐ Pruebas de integración de APIs
- ☐ Testing de usabilidad con usuarios beta
- ☐ Optimización de performance (Lighthouse)
- ☐ Compresión de assets

Semana 3:

- ☐ Configurar Vercel/Netlify
- ☐ Configurar Railway para backend
- ☐ Variables de entorno en producción
- ☐ Despliegue a staging
- ☐ Testing en producción

Semana 4:

- ☐ Corrección de bugs finales
- ☐ Optimización de SEO
- ☐ Configurar Analytics
- ☐ Deploy a producción
- ☐ Monitoreo inicial

Entregable: Aplicación en producción

Fase 5: Documentación y Lanzamiento

Semana 1-2:

- ☐ Documentación técnica final
- ☐ Manual de usuario
- ☐ Guía de contribución (si open source)
- ☐ README completo
- ☐ Comentarios en código

Semana 3:

- ☐ Crear material promocional
- ☐ Preparar posts para redes sociales
- ☐ Video demo de la aplicación
- ☐ Landing page (opcional)

Semana 4:

- ☐ Lanzamiento oficial

- [] Campaña en redes sociales
 - [] Recolección de feedback
 - [] Análisis de métricas iniciales
 - [] Informe final del proyecto
- Entregable: Proyecto completo + Informe final

9.4 Gestión de Riesgos - Plan de Mitigación

Riesgo 1: API no disponible o cambios en términos

Probabilidad: Media (30%)

Impacto: Alto

Plan de Mitigación:

- Tener API alternativa identificada para cada servicio
- Implementar fallback a datos cacheados
- Sistema de alertas cuando API falla

Plan de Contingencia:

```
javascript
async function getWeatherWithFallback() {
  try {
    return await openWeatherAPI();
  } catch {
    try {
      return await weatherAPIAlternative();
    } catch {
      return getCachedWeather();
    }
  }
}
```

Riesgo 2: Retraso en desarrollo

Probabilidad: Alta (60%)

Impacto: Medio

Plan de Mitigación:

- Desarrollo basado en prioridades MoSCoW
- MVP primero, features adicionales después
- Buffer de 2 semanas en cronograma

Acciones:

- Si retraso <2 semanas: Posponer features "Could Have"
- Si retraso >2 semanas: Eliminar módulo Eventos del MVP

Riesgo 3: Problemas de performance

Probabilidad: Baja (20%)

Impacto: Alto

Plan de Mitigación:

- Testing de performance desde Fase 2
- Lazy loading de componentes
- Optimización de imágenes
- Code splitting

Métrica objetivo: Lighthouse Score >90

Riesgo 4: Baja adopción de usuarios

Probabilidad: Media (40%)

Impacto: Medio

Plan de Mitigación:

- Plan de marketing desde Fase 1
- Identificar early adopters
- Feedback loops constantes
- Iteraciones basadas en uso real

Meta mínima: 50 usuarios activos (50% de objetivo)

10. ANEXOS

10.1 Glosario de Términos

Término	Definición
AQI/ICA	Air Quality Index / Índice de Calidad del Aire
API	Application Programming Interface
CDN	Content Delivery Network
CORS	Cross-Origin Resource Sharing
GeoJSON	Geographic JSON format
HMR	Hot Module Replacement
MVP	Minimum Viable Product
ORM	Object-Relational Mapping
PWA	Progressive Web App
SPA	Single Page Application
TTL	Time To Live (tiempo de caché)
UI	User Interface
UX	User Experience
WCAG	Web Content Accessibility Guidelines

10.2 Referencias y Recursos

APIs Documentación

- [OpenWeatherMap API Docs](https://openweathermap.org/api)

- [AQICN API Documentation](https://aqicn.org/api/)
- [Eventbrite API](https://www.eventbrite.com/platform/api)
- [Leaflet Documentation](https://leafletjs.com/)

Datos Abiertos Bogotá

- [IDECA - Infraestructura de Datos](https://www.ideca.gov.co/)
- [Datos Abiertos Bogotá](https://datosabiertos.bogota.gov.co/)

Tecnologías

- [React Documentation](https://react.dev/)
- [TailwindCSS Docs](https://tailwindcss.com/docs)
- [Vite Guide](https://vitejs.dev/guide/)

Diseño

- [Material Design Guidelines](https://m3.material.io/)
- [WCAG 2.1 Guidelines](https://www.w3.org/WAI/WCAG21/quickref/)

10.3 Herramientas de Prototipado

Link a Figma:

<https://www.figma.com/site/vBQGIFM07oMjilCzESFYgz/CIVICVIEWWEB?node-id=0-1&t=WkwHFa8Lg75gNSny-1>

10.4 Métricas de Éxito

Métricas Técnicas (Fase 4)

Métrica	Objetivo	Herramienta
Lighthouse Performance	>90	Chrome DevTools
First Contentful Paint	<1.5s	Lighthouse
Time to Interactive	<3s	Lighthouse
Bundle Size	<200KB	Vite Build Analyzer
API Response Time 	<2s	Network Tab
Uptime	>99%	UptimeRobot

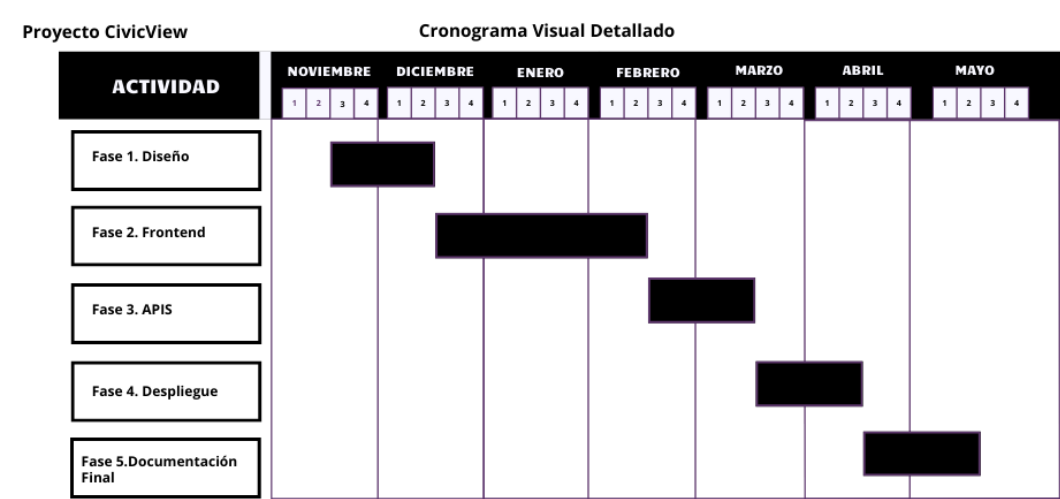
Métricas de Usuario (Fase 5)

Métrica	Objetivo	Herramienta
Usuarios Activos (Mes 1)	100+	Google Analytics
Tasa de Retención	>40%	GA4
Tiempo Promedio Sesión	1-2 min	GA4
Bounce Rate 	<40%	GA4
Page Views/Session	>2	GA4

Métricas de Negocio (Post-lanzamiento)

Métrica	Objetivo Año 1	Método
Usuarios Mensuales	1,000-5,000	Analytics
NPS (Net Promoter Score)	>40	Encuestas
Tiempo ahorrado por usuario	8+ min/día	Estudio UX
Costo por usuario	<\$500 COP	Análisis financiero

10.5 Cronograma Visual Detallado



- Hitos Clave:

 -  17 Dic 2025: Entrega Fase 1 (ESTE DOCUMENTO)
 -  17 Feb 2026: Frontend funcional
 -  17 Marzo 2026: Integración completa
 -  17 Abril 2026: Aplicación en producción
 -  17 Mayo 2026: Lanzamiento oficial + Informe final

10.6 Lista de Verificación de Calidad

Antes de cada commit

- [] Código formateado con Prettier
- [] Sin errores de ESLint
- [] Nombres de variables descriptivos
- [] Comentarios en funciones complejas
- [] Sin console.logs en producción

Antes de cada merge a main

- [] Todas las pruebas pasan
- [] Build de producción exitoso
- [] Revisión de código (self-review)
- [] Actualizar CHANGELOG.md
- [] Documentación actualizada

Antes del deploy a producción

- [] Testing en staging completo
- [] Variables de entorno configuradas
- [] Backup de base de datos
- [] Plan de rollback preparado
- [] Monitoreo configurado

CONCLUSIONES DE FASE 1

Logros de esta Fase

- ✓ ****Especificación completa de requisitos**** funcionales y no funcionales
- ✓ ****Arquitectura técnica definida**** con diagramas y justificación
- ✓ ****Diseño de base de datos**** normalizado y optimizado
- ✓ ****Investigación exhaustiva de APIs**** con alternativas identificadas
- ✓ ****Experiencia de usuario documentada**** con wireframes y flujos
- ✓ ****Sistema de diseño completo**** (colores, tipografía, componentes)
- ✓ ****Stack tecnológico seleccionado**** con criterios objetivos
- ✓ ****Plan de implementación detallado**** con mitigación de riesgos

Próximos Pasos Inmediatos

- 1. Revisar y aprobar este documento con el coordinador académico
- 2. Completar prototipos en Figma (alta fidelidad)
- 3. Preparar entorno de desarrollo (instalar herramientas)
- 4. Iniciar Fase 2 según cronograma (1 Nov 2025)

Reflexión del Desarrollador

Esta fase ha establecido bases sólidas para el proyecto. La arquitectura propuesta es escalable, las tecnologías son apropiadas para el alcance, y el diseño está centrado en resolver el problema real del usuario: ****ahorrar tiempo consultando información fragmentada.**

El mayor reto identificado es la integración efectiva de múltiples APIs externas manteniendo performance y confiabilidad. La estrategia de caché multi-nivel y fallbacks debería mitigar este riesgo.

El proyecto está listo para pasar a la fase de desarrollo. Con 180 horas invertidas en planificación, tenemos un roadmap claro y alcanzable para las siguientes 684 horas.

APROBACIONES

Rol	Nombre	Firma	Fecha
Desarrollador	Jeyson Andrés Ortiz Mendoza		17/12/2025
Coordinador Académico	Eileen Karina Castañeda Losada		17/12/2025
Instructor de seguimiento	Hector David Toledo Garcia		17/12/2025

FIN DEL DOCUMENTO

Versión 1.0 - Diciembre 2025

CivicView: Plataforma Web Integrada de Información Ciudadana en Tiempo Real*

SENA - Centro Agroempresarial y Desarrollo Pecuario del Huila*